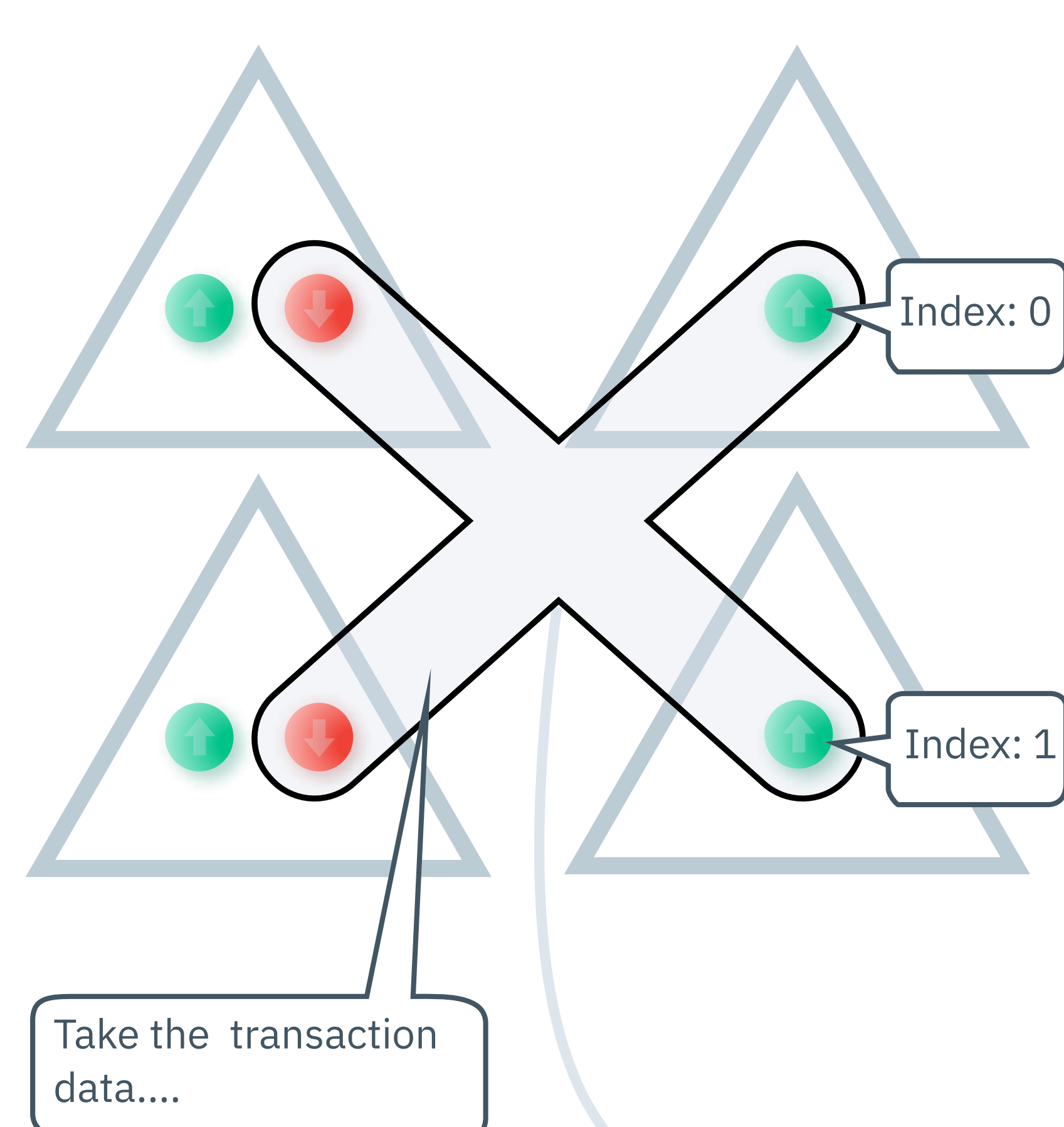


Cerberus - How Radix achieves infinite linear scalability while preserving atomic composability

Shard Allocation

8

So we've previously mentioned that each substate has its own unique shard in the shardspace. So how does this work? How does a substate get allocated its own unique shard?



The transaction to the left is composed of two shut down substates and two bring up substates. The transaction will also include data such as a header and footer, and cryptographic signatures authorizing the transaction, e.g. from the people who "own" the tokens that the substate represents. We cover transactions in more depth in the next episode.

Each of the bring up substates within the transaction has an index position. The first bring up substate is Index: 0, the second - Index: 1, and so forth.

Each of these bring up substates will need their own address in the shardspace. Shut down substates don't need a new shard address as they live with a bring up substate from a previous transaction.

So to start with, we must first generate a unique identifier for the transaction. This is done by taking the transaction data, and hashing it. The output is a hash that is 256 bits (78 digits) long. This number serves as that transaction's unique identifier - its Transaction ID.

We can then add the index number of the bring up substate to the Transaction ID, which creates a number that is unique to that substate. So in this example, the substate has Index: 1.

We can then apply the hash again to the combined data, to generate a unique 256 bit substate ID.

Hashing algorithms are deterministic. Given the same input data, you will always get the same output data.

256 bit #

...then hash that transaction data...

...and you get a 78 digit TX ID...

Transaction ID: 657,376,112,198,877,997,645, 34

Index: 1

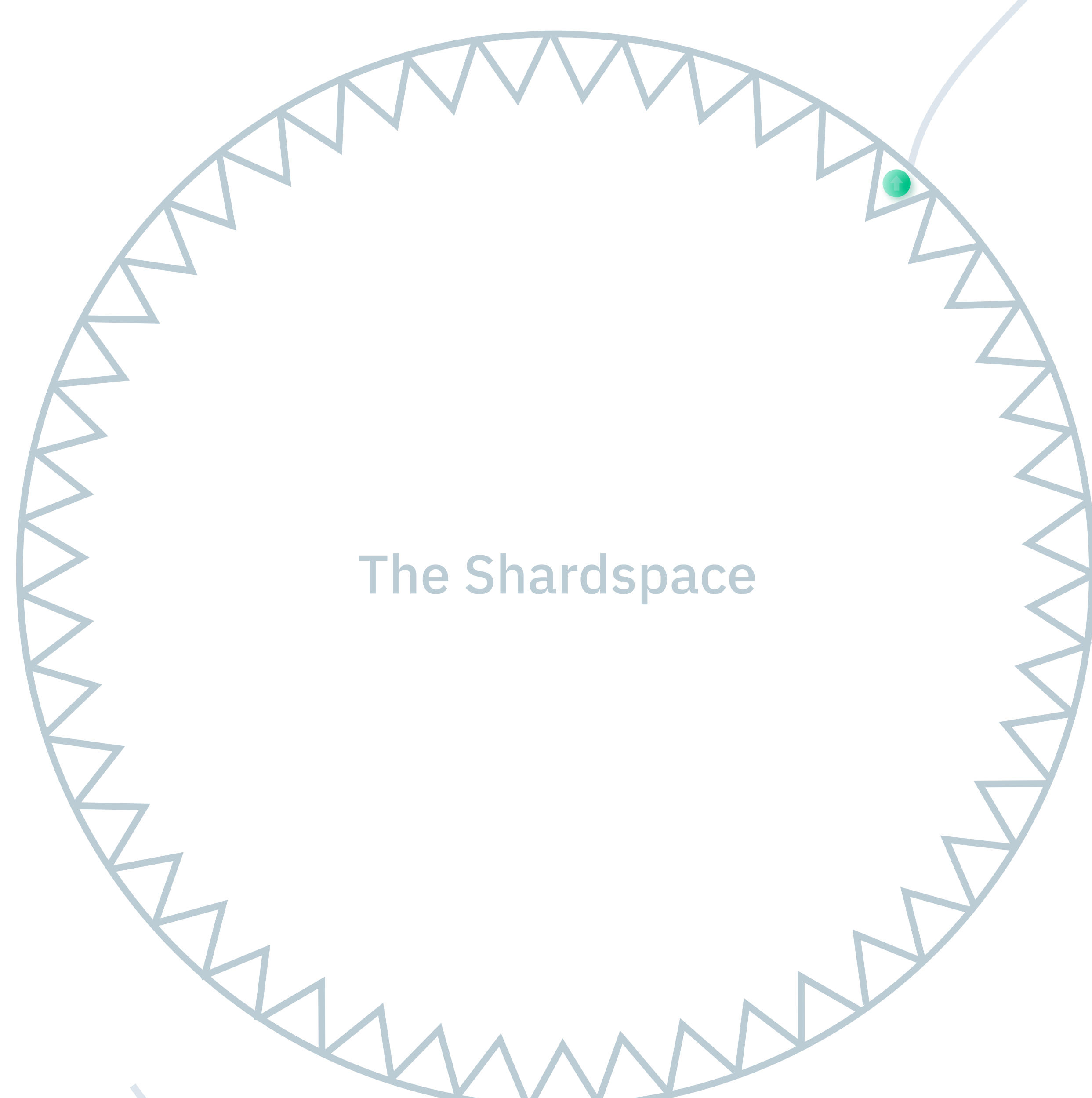
...to get a unique ID for a substate, you add that substate's index within the transaction...

256 bit #

...hash that...

Substate ID: 144,826,489,488,222,149,009.878.7

...and you get a unique 78 digit ID for that substate.



The number of possible substate IDs maps perfectly to the 2^{256} shardspace as they're both 256 bit. In other words, the substate ID is that substate's shard address. The two are inextricably linked.

This means that just by looking at a substate, any piece of software can instantly determine what shard it belongs in. The substate can't live anywhere else.

Substates are scattered across the shardspace as Substate IDs can be expected to be nearly uniformly distributed across the shardspace. This "load balances" the computational requirements across the shardspace.

Because the shardspace is so large, the chances of there being a "collision" is practically zero. Even at a trillion transactions per second, we would have entered the "degenerate" era of the universe (1,000 trillion years from now); where only brown dwarfs, white dwarfs, neutron stars, and black holes remain; prior to there being a $\sim 0.0000001\%$ chance of a single shard collision during that entire period [1]. And in the extremely unlikely event that there is a collision, the transaction would just fail.

Additionally, the potential for attack or disruption by an adversary taking over any single shard is low, as a single shard only ever contains a single substate.

Finally, as the shardspace is fixed-size, static, and well-known ahead of time, no index is ever required, as every substate has its own fixed shard. This removes the need for complex dynamic sharding procedures that other sharded networks have, as when they try and scale by adding new shards (such as new blockchains that run in parallel), lots of communication is often required redefining where everything lives.

This unique approach is a key part of what enables Cerberus to process an almost unlimited number of parallel transactions, as Cerberus will never run out of shards.

So now we know what substates are, how they're created, and how they're allocated a shard address, but what does an actual transaction look like?

[1] Trillion = 10^{12} ; seconds in year = 3×10^7 ; degenerate era = 10^{15} years

Therefore $(10^{12}) \times (3 \times 10^7) \times (10^{15}) = 3 \times 10^{34}$ total transactions at a trillion TPS until the [degenerate era of the universe](#)

Using a "[birthday attack](#)", $\sim 0.0000001\%$ (10^{-9}) odds for a collision in a 2^{256} shardspace occur after 1.5×10^{34} hashes

The version of Cerberus described in this infographic series is scheduled to launch as part of the fully sharded Radix Xi'an release. Please visit www.radixdlt.com for details on the Radix roadmap.